

```

In [40]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.api import VAR

Q1_END = pd.Timestamp("2025-03-31") # Last date of Q1 2025
Q2_START = pd.Timestamp("2025-04-01") # Start of Q2 2025
Q2_END = pd.Timestamp("2025-06-30") # End of Q2 2025
H = 8

def get_A_and_c(res):

    A = [res.coefs[j] for j in range(res.k_ar)] #
    c = res.intercept.copy()
    return A, c

def simulate_path(A, c, y_hist, shocks):
    p = len(A)
    k = y_hist.shape[1]
    Y = y_hist.copy()
    out = []
    for h in range(shocks.shape[0]):
        y_next = c.copy()
        for j in range(1, p + 1):
            y_next += A[j - 1] @ Y[-j, :]
        y_next += shocks[h, :]
        out.append(y_next)
        Y = np.vstack([Y, y_next])[-p:, :]
    return np.vstack(out)

gdp_q1 = gdp_block_growth.loc[:Q1_END].dropna()
sel_q1 = VAR(gdp_q1).select_order(maxlags=8)
lag_opt_q1 = int(sel_q1.aic)
print(f"Optimal lag based on Q1 data (AIC criterion): p = {lag_opt_q1}")

var_q1 = VAR(gdp_q1).fit(lag_opt_q1)
A_q1, c_q1 = get_A_and_c(var_q1)

gdp_q2 = gdp_block_growth.loc[
    (gdp_block_growth.index >= Q2_START) & (gdp_block_growth.index <= Q2_END)
].dropna()

Y_hist = gdp_q1.iloc[-lag_opt_q1:, :].to_numpy()
q2_resids = []
Y = Y_hist.copy()

for t in range(len(gdp_q2)):
    y_hat = c_q1.copy()
    for j in range(1, lag_opt_q1 + 1):

```

```

        y_hat += A_q1[j - 1] @ Y[-j, :]
    u_t = gdp_q2.iloc[t, :].to_numpy() - y_hat # residual = actual - fitted
    q2_resids.append(u_t)
    Y = np.vstack([Y, gdp_q2.iloc[t, :].to_numpy()])[-lag_opt_q1:, :]

q2_resids = np.vstack(q2_resids)
q2_shock_mean = q2_resids.mean(axis=0)
print("\nMean shock vector for Q2:")
print(pd.Series(q2_shock_mean, index=gdp_q1.columns))

start_period = gdp_q1.index.to_period("Q")[-1]
forecast_index = pd.period_range(start_period + 1, periods=H, freq="Q")

baseline_shocks = np.zeros((H, len(gdp_q1.columns)))
y_hist = gdp_q1.iloc[-lag_opt_q1:, :].to_numpy()
f_base = simulate_path(A_q1, c_q1, y_hist, baseline_shocks)

shift_shocks = np.tile(q2_shock_mean, (H, 1))
f_shift = simulate_path(A_q1, c_q1, y_hist, shift_shocks)

forecast_q1_baseline = pd.DataFrame(f_base, index=forecast_index, columns=gdp_q1.columns)
forecast_q1_shifted = pd.DataFrame(f_shift, index=forecast_index, columns=gdp_q1.columns)

print("\nForecasts complete:")
print("Baseline (Q1 model, no shocks) and Parallel-shift (Q1 model + Q2 shocks) generated")

target_col = gdp_q1.columns[0]
plt.figure(figsize=(9, 4.5))

forecast_q1_baseline.index = forecast_q1_baseline.index.to_timestamp()
forecast_q1_shifted.index = forecast_q1_shifted.index.to_timestamp()

plt.plot(forecast_q1_baseline[target_col], label="Baseline (Q1 model)", marker='o')
plt.plot(forecast_q1_shifted[target_col], label="Parallel-shift (+Q2 shocks)", marker='o')

plt.title(f"VAR(p={lag_opt_q1}) Forecast Comparison - {target_col}")
plt.xlabel("Quarter")
plt.ylabel("Growth rate")
plt.legend()
plt.tight_layout()
plt.show()

```

Optimal lag based on Q1 data (AIC criterion): $p = 1$

Mean shock vector for Q2:

Estimates

Final consumption expenditure	0.007355
Gross fixed capital formation	-0.003123
General governments final consumption expenditure	0.009305
Exports of goods and services	-0.084595
Less: imports of goods and services	-0.018803

dtype: float64

Forecasts complete:

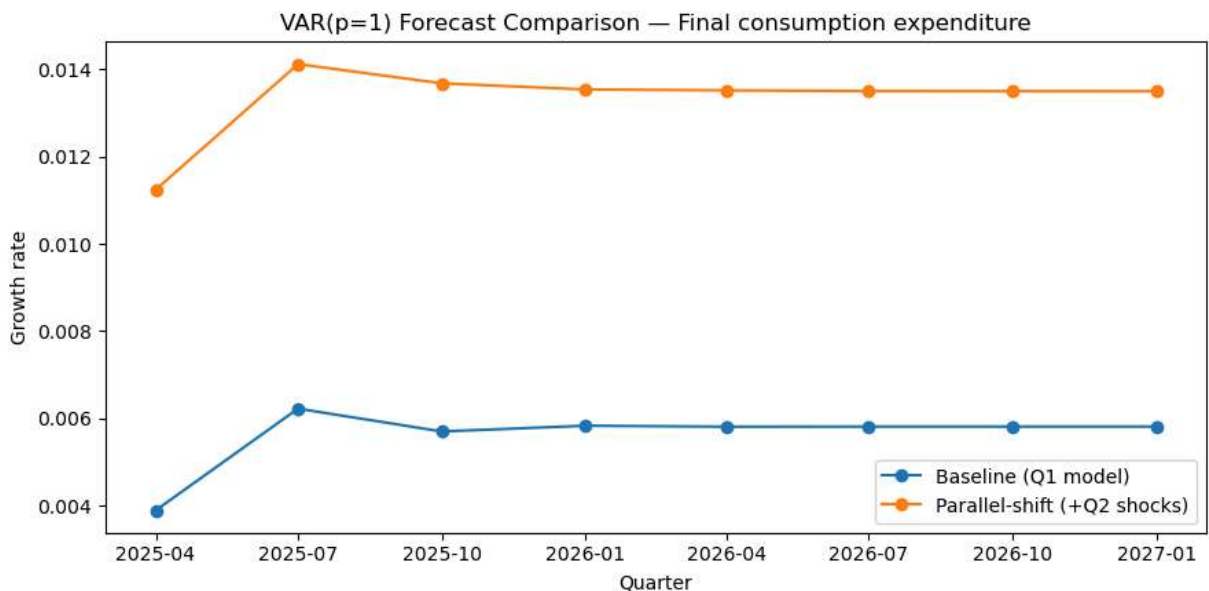
Baseline (Q1 model, no shocks) and Parallel-shift (Q1 model + Q2 shocks) generated successfully.

c:\Users\logc0\anaconda3\envs\hw6\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency QS-OCT will be used.

self._init_dates(dates, freq)

c:\Users\logc0\anaconda3\envs\hw6\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency QS-OCT will be used.

self._init_dates(dates, freq)



```
In [39]: target_col = "Final consumption expenditure"
use_shifted = True # True= Parallel-shift(+Q2 shocks), False= Baseline

def _to_dt_index(idx):
    return idx.to_timestamp() if isinstance(idx, pd.PeriodIndex) else pd.DatetimeIndex

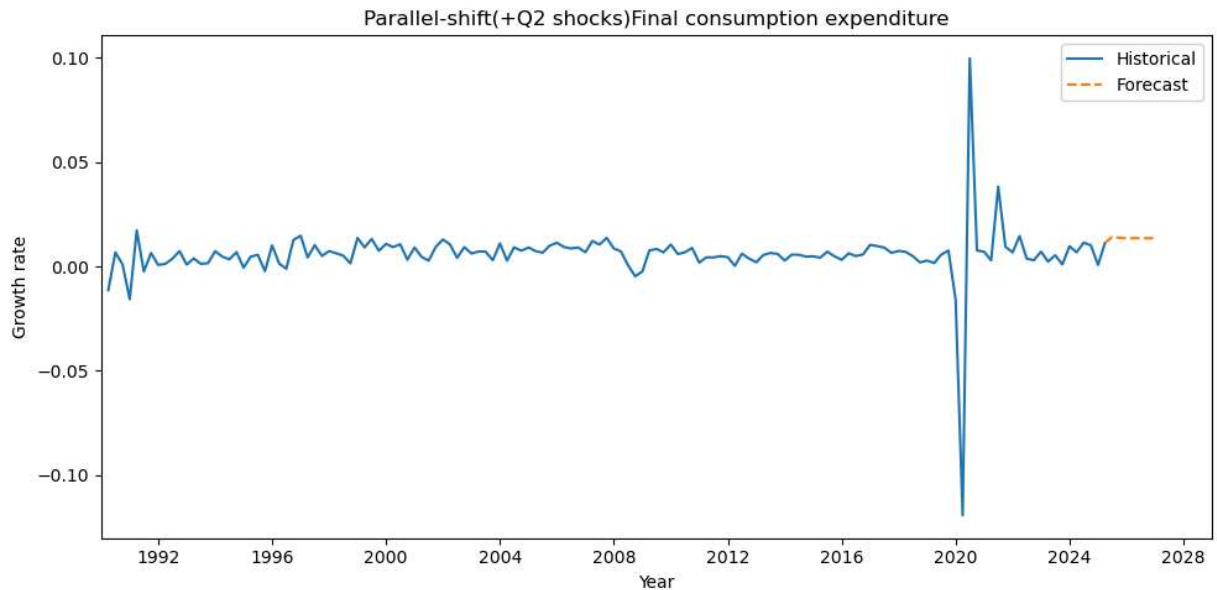
hist_series = gdp_block_growth[target_col].copy()
hist_series.index = _to_dt_index(hist_series.index)

fc_df = forecast_q1_shifted if use_shifted else forecast_q1_baseline
forecast_series = fc_df[target_col].copy()
forecast_series.index = _to_dt_index(forecast_series.index)

plt.figure(figsize=(10, 5))
plt.plot(hist_series, label="Historical")
```

```
plt.plot(forecast_series, linestyle="--", label="Forecast")

plt.title(f"Parallel-shift(+Q2 shocks){target_col}")
plt.xlabel("Year")
plt.ylabel("Growth rate")
plt.xlim(pd.Timestamp("1990-01-01"), pd.Timestamp("2028-12-31"))
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [14]: import matplotlib.pyplot as plt

for col in forecast_df.columns:
    plt.figure(figsize=(8,4))
    plt.plot(gdp_block_growth[col], label='Historical')
    plt.plot(forecast_df[col], label='Forecast', linestyle='--')
    plt.title(f"Forecast for {col}")
    plt.legend()
    plt.show()
```

